# Geo API Reference

**Version 1.0**
**Geometric Language Reasoning Engine**

> Reasoning is a $C^2$-continuous path through semantic space.
> Memory is a three-tier NURBS/B-Spline/Bézier manifold.
> Intelligence is geometry. Language is output only.

GeoJS is a deterministic, auditable, and portable JavaScript library that implements the **Hybrid Geometric AI Architecture** described in the white paper *"Hybrid Geometric AI Architecture for Financial Modeling and Hybrid AI:Human Business Teams"*. It replaces statistical black-boxes with a transparent geometric foundation for AI reasoning, memory, and human-AI collaboration.

---

## Overview: The Unifying AI Theory (FOBBs)

GeoJS operationalizes the **FOBBs Unifying Theory**:

- **Comprehension** – Parse raw input into a geometric structure by identifying high-curvature events (knots) and assigning them to the appropriate memory tier (NURBS, B-Spline, or Bézier).
- **Orchestration** – Navigate the geometric memory manifold by sampling only the curve segments relevant to the current task, avoiding full context loading.
- **Reasoning** – Find a $C^2$-continuous path between semantic points (premise → conclusion) while being attracted to high-weight NURBS control points (past significant events) and biased by the strategic Bézier "North Star".
- **Evaluation** – Assess reasoning quality via curvature analysis; high curvature signals uncertainty and triggers human escalation.

All memory is stored in a **three-tier manifold**:

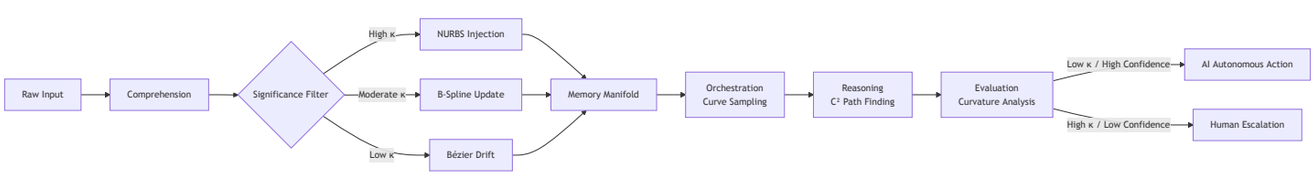
| Tier | Curve Type | Time Horizon | Weight Structure | Use Case |
|---|---|---|---|---|
| Tier 1 | NURBS | 0–12 months | Heterogeneous, significance-driven | Real-time reasoning, crisis response |
| Tier 2 | B-Spline | 13–60 months | Normalized, uniform | Operational context, trend identification |
| Tier 3 | Bézier | Full fund life | Fixed, strategic | Long-horizon orientation, LP reporting |

The manifold is **navigable**, **auditable**, and **portable** via the **Memory Manifest** – a JSON serialisation that can be exported, imported, and used as a "Skill Patch" to instantly transfer expertise between GeoJS instances.

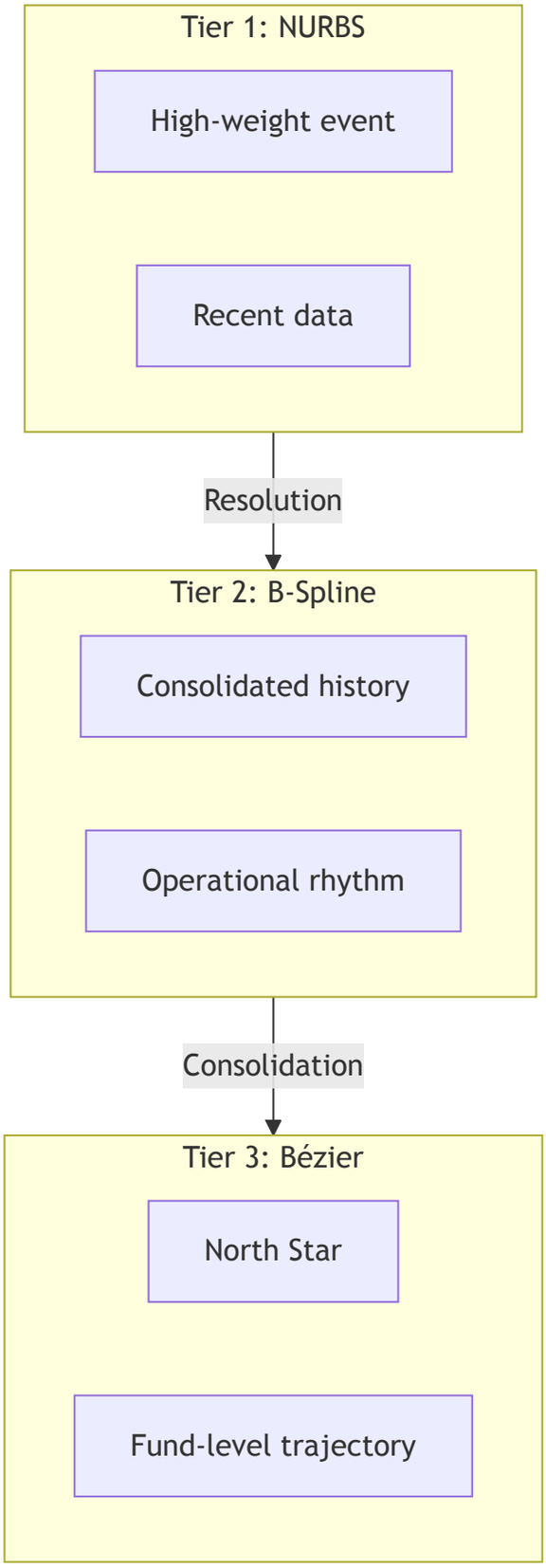
---

## Mermaid Diagrams

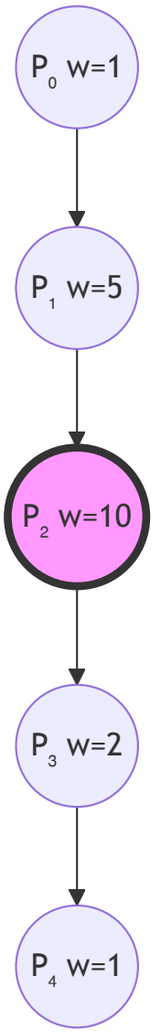
### FOBBs Pipeline




### Three-Tier Memory Manifold




**NURBS Curve with Weighted Control Points**




---

## Core Concepts

- **Semantic Space** – A continuous, multi-dimensional coordinate system where each dimension represents a measurable attribute (e.g., risk, liquidity, time). The number of dimensions is fixed per manifold.
- **SemanticPoint** – A point in semantic space, with coordinates, a human-readable label, a domain, a weight (significance), and a timestamp.
- **Control Points** – The points that define the shape of a NURBS, B-Spline, or Bézier curve. In GeoJS they are `SemanticPoint` instances.
- **Knot Vector** – A non-decreasing sequence of parameter values that partitions the curve domain and controls the influence of control points.
- **Curvature κ** – A measure of how sharply the reasoning path bends; high curvature indicates high uncertainty and triggers human escalation.
- **North Star** – A quadratic Bézier curve that represents the strategic long-term objective. It biases all reasoning paths.

---

## Class: `GeoJS`

The main engine. Create an instance with a configuration object.

### Constructor

```
new GeoJS(config)
```

**Parameters**

| Name | Type | Description |
|---|---|---|
| `config.dimensions` | `number` | Dimensionality of the semantic space (default `7`). |
| `config.degree` | `number` | Spline degree (default `3` – cubic, required for $C^2$ continuity). |
| `config.domain` | `string` | Domain namespace (default `'general'`). |
| `config.thresholds` | `Object` | Curvature thresholds for autonomy levels. Default: `{ low: 0.15, moderate: 0.35, high: 0.65 }`. |

| | | |
|---|---|---|
| `config.northStar` | `Object` | Initial strategic Bézier control points: `{ origin, focus, goal }` each with a `label` and `measurements`. |

**Example**

```
const geo = new GeoJS({
  dimensions: 7,
  degree: 3,
  domain: 'infrastructure-finance',
  thresholds: { low: 0.1, moderate: 0.3, high: 0.6 },
  northStar: {
    origin: { label: 'Fund inception', measurements: { risk: 0.7, liquidity: 0.5, time: 0.0 } },
    focus: { label: 'Current operations', measurements: { risk: 0.5, liquidity: 0.7, time: 0.5 } },
    goal: { label: 'Year 5 exit', measurements: { risk: 0.2, liquidity: 0.9, time: 1.0 } }
  }
});
```

## Public Methods

### `encode(concept)`

Maps a human-readable concept to a `SemanticPoint` in the manifold's coordinate space.

**Parameters**

| Name | Type | Description |
|---|---|---|
| `concept.label` | `string` | Human-readable name. |
| `concept.measurements` | `Object` | Named dimension values (0–1). Keys must match the domain schema. |
| `concept.domain` | `string` | (Optional) Domain override. |
| `concept.weight` | `number` | (Optional) Initial significance weight. |

**Returns** `SemanticPoint` – a point in semantic space.

**Example**

```
const point = geo.encode({
  label: 'Covenant breach',
  measurements: { risk: 0.95, liquidity: 0.3, time: 2.5 },
  weight: 10.0
});
```

---

### `evaluate(t)`

Evaluates the NURBS manifold at parameter `t` (0–1) and returns the semantic point at that position. Uses the Cox–de Boor algorithm with rational basis.

**Parameters**

| Name | Type | Description |
|---|---|---|
| `t` | `number` | Parameter in [0, 1]. |

**Returns** `SemanticPoint | null` – the interpolated point, or `null` if no control points exist.

---

### `reason(from, to, options)`

Finds a $C^2$-continuous reasoning path from one semantic point to another. The path is attracted to high-weight NURBS control points and biased by the North Star.

**Parameters**

| Name | Type | Description |
|---|---|---|
| `from` | `SemanticPoint` | Premise (start point). |
| `to` | `SemanticPoint` | Conclusion (end point). |
| `options.steps` | `number` | Number of intermediate steps (default `10`). |
| `options.maxIter` | `number` | Maximum repair iterations (default `100`). |

**Returns** `ReasoningPath` – an object containing the path steps, curvature, confidence, and continuity flag.

**Returns** `ReasoningPath` – an object containing the path steps, curvature, confidence, and continuity flag.

**Throws** `GeometricContinuityError` if the path cannot be made C²-continuous.

**Example**

```
const from = geo.encode({ label: 'Current DSCR', measurements: { dscr: 1.2 } });
const to = geo.encode({ label: 'Covenant threshold', measurements: { dscr: 1.0 } });
const path = geo.reason(from, to, { steps: 20 });
if (path.continuous) {
  console.log(`Path curvature: ${path.curvature}`);
}
```

---

### `inject(point, weight)`

Inserts a high-significance event into the manifold using Boehm's knot insertion algorithm, preserving C² continuity. The point is assigned to the appropriate memory tier based on its weight.

**Parameters**

| Name | Type | Description |
|---|---|---|
| `point` | `SemanticPoint` | The event to inject. |
| `weight` | `number` | Significance weight (1–10). Default `1.0`. |

**Returns** `GeoJS` (chainable).

**Notes**

- Weight ≥ 5.0 → NURBS tier (active, high-fidelity).
- Weight 2.0–4.9 → B-Spline tier (historical).
- Weight < 2.0 → updates the North Star (strategic drift).

---

### `resolve(fromTier, toTier)`

Moves points from a higher-resolution tier to a lower-resolution tier when they age out of the active window. This is not compression but resolution reduction based on current relevance.

**Parameters**

| Name | Type | Description |
|---|---|---|
| `fromTier` | `string` | `'nurbs'` or `'bspline'`. |
| `toTier` | `string` | `'bspline'` or `'bezier'`. |

**Returns** `GeoJS` (chainable).

---

### `exportManifest(resolution)`

Exports the current state of the manifold as a **Memory Manifest** JSON object. The manifest can be signed, stored, and later imported.

**Parameters**

| Name | Type | Description |
|---|---|---|
| `resolution` | `string` | `'nurbs'` (full fidelity), `'bspline'` (lossy), or `'bezier'` (strategic only). |

**Returns** `Object` – the signed Memory Manifest.

---

### `importManifest(manifest)`

Imports a Memory Manifest, injecting its control points into the specified tier and updating the North Star if present. Verifies signature and coordinate space compatibility.

**Parameters**

| Name | Type | Description |
|---|---|---|
| `manifest` | `Object` | A valid Memory Manifest (from `exportManifest`). |

**Returns** `GeoJS` (chainable).

**Throws** `ManifestCompatibilityError` or `ManifestIntegrityError` on mismatch or invalid signature.

---

### `on(event, handler)`

Subscribes to geometric events, especially curvature threshold crossings and escalation requests.

**Parameters**

| Name | Type | Description |
|---|---|---|
| `event` | `string` | One of: `'curvature:low'`, `'curvature:moderate'`, `'curvature:high'`, `'curvature:undefined'`, `'continuity:violated'`, `'escalation:required'`. |
| `handler` | `Function` | Callback that receives an event data object. |

**Returns** `GeoJS` (chainable).

**Example**

```
geo.on('escalation:required', (data) => {
  console.log(`Human needed! Curvature: ${data.κ}`);
  notifyRiskManager(data.path);
});
```

---

### `comprehend(input)`

Implements the **Comprehension** function: parses raw input, computes its local curvature relative to the current manifold, and assigns a memory tier and weight.

**Parameters**

| Name | Type | Description |
|---|---|---|
| `input` | `Object` | Same structure as `encode` (label, measurements). |

**Returns** `Object` with properties:

- `point` – the encoded `SemanticPoint`.
- `tier` – `'nurbs'`, `'bspline'`, or `'bezier'`.
- `weight` – assigned weight (10, 5, or 1).
- `curvature` – computed local curvature.

---

### `orchestrate(taskDomain, t)`

Implements the **Orchestration** function: returns the control points most relevant to a task at parameter position `t`. Includes all points near `t` and any high-weight NURBS points (weight ≥ 8) regardless of position.

**Parameters**

| Name | Type | Description |
|---|---|---|
| `taskDomain` | `string` | Domain of the current task (e.g., `'solar'`). |
| `t` | `number` | Parameter position in [0, 1]. |

**Returns** `SemanticPoint[]` – array of relevant control points.

---

### `evaluatePath(path)`

Implements the **Evaluation** function: determines whether the reasoning path is safe for autonomous action based on its curvature and confidence.

**Parameters**

| Name | Type | Description |
|---|---|---|
| `path` | `ReasoningPath` | The path to evaluate. |

**Returns** `Object` with:

- `autonomous` – `boolean` (true if curvature < moderate threshold AND confidence > 0.7).
- `curvature` – `number`.
- `confidence` – `number` (based on local knot density).

- `confidence` – `number` (based on local knot density).

If `autonomous` is false, an `'escalation:required'` event is emitted.

---

### Public Attributes

| Name | Type | Description |
|---|---|---|
| `dimensions` | `number` | Dimensionality of the semantic space (read-only). |
| `degree` | `number` | Spline degree (read-only). |
| `domain` | `string` | Domain namespace (read-only). |
| `thresholds` | `Object` | Current curvature thresholds (can be updated). |

---

## Class: `SemanticPoint`

Represents a point in semantic space.

**Constructor** `new SemanticPoint({ coordinates, label, domain, weight, timestamp })`

**Properties** (all read-only once constructed)

| Name | Type | Description |
|---|---|---|
| `coordinates` | `Float64Array` | Position in semantic space. |
| `label` | `string` | Human-readable name. |
| `domain` | `string` | Domain namespace. |
| `weight` | `number` | Significance weight (1–10). |
| `timestamp` | `number` | Creation time (ms since epoch). |

---

## Class: `ReasoningPath`

Returned by `reason()`. Represents a $C^2$-continuous path through semantic space.

**Constructor** (internal – not called directly)

**Properties**

| Name | Type | Description |
|---|---|---|
| `steps` | `SemanticPoint[]` | Intermediate points along the path. |
| `from` | `SemanticPoint` | Start point. |
| `to` | `SemanticPoint` | End point. |
| `curvature` | `number` | Maximum curvature along the path. |
| `confidence` | `number` | Confidence based on local knot density. |
| `continuous` | `boolean` | True if $C^2$ continuity was maintained. |
| `domain` | `string` | Domain of the path. |
| `timestamp` | `number` | Creation time. |

**Methods**

`toAuditTrail()`

Exports the path as a human-readable audit trail, listing each step with its parameter `t`, label, coordinates, and domain.

**Returns** `Array<Object>` – step-by-step audit entries.

---

## Appendix: The Math

### Bézier Curves

A Bézier curve of degree $n$ is defined by $n+1$ control points $P_0, P_1, \ldots, P_n$ and the Bernstein basis:

A Bézier curve of degree `n` is defined by `n+1` control points `P₀, P₁, …, Pₙ` and the Bernstein basis:

$$\mathbf{C}(t) = \sum_{i=0}^{n} \binom{n}{i} t^i (1-t)^{n-i} \mathbf{P}_i, \quad t \in [0,1]$$

**Properties**

- Global influence: moving any control point affects the entire curve.
- Used in GeoJS for the strategic **North Star** (quadratic Bézier, n=2).
- The North Star biases all reasoning paths via a deviation penalty.

## B-Splines

A B-Spline generalises Bézier curves by partitioning the parameter domain into segments. It is defined by a knot vector `T = {t₀, t₁, …, tₘ}` and basis functions `Nᵢ,ₖ(t)` of degree `k`:

$$\mathbf{C}(t) = \sum_{i=0}^{n} \mathbf{P}_i N_{i,k}(t)$$

The Cox–de Boor recursion defines the basis:

- Degree 1: `Nᵢ,₁(t) = 1` if `tᵢ ≤ t < tᵢ₊₁`, else `0`.
- Higher degrees:

$$N_{i,k}(t) = \frac{t - t_i}{t_{i+k-1} - t_i} N_{i,k-1}(t) + \frac{t_{i+k} - t}{t_{i+k} - t_{i+1}} N_{i+1,k-1}(t)$$

**Properties**

- **Local control**: modifying `Pᵢ` affects only the curve in the interval `[tᵢ, tᵢ₊ₖ]`.
- GeoJS uses cubic B-Splines (k=3) for the **operational memory tier**.

## NURBS (Non-Uniform Rational B-Splines)

NURBS extend B-Splines by assigning a weight `wᵢ` to each control point, making the curve rational:

$$\mathbf{C}(t) = \frac{\sum_{i=0}^{n} w_i \mathbf{P}_i N_{i,k}(t)}{\sum_{i=0}^{n} w_i N_{i,k}(t)}$$

**Properties**

- A high weight creates a “gravity well” – the curve is pulled toward that control point.
- Non-uniform knot spacing allows sharp transitions (kinks) at specific parameter values.
- GeoJS uses NURBS for the **active memory tier**, where high-weight points represent significant events (covenant breaches, sudden market moves).

## Boehm’s Knot Insertion

To insert a new knot `t*` into a B-Spline/NURBS while preserving the exact curve shape (and C² continuity), Boehm’s algorithm computes new control points for the affected span:

$$\alpha_i = \frac{t^* - T_i}{T_{i+p-1} - T_i}$$

New points `Q_i` are linear combinations of original points `P_{i-1}` and `P_i`:

$$Q_i = \alpha_i P_i + (1 - \alpha_i) P_{i-1}$$

GeoJS implements a non-in-place version that rebuilds the control point array after insertion.

## Curvature in Semantic Space

For a parametric curve `C(t)`, the curvature `κ` (independent of parameterisation) is:

$$\kappa = \frac{\|C'(t) \times C''(t)\|}{\|C'(t)\|^3}$$

In `n`-dimensional space, the cross product magnitude is generalised to the Frobenius norm of the skew-symmetric matrix:
$$\|C' \times C''\| = \sqrt{\sum_{i<j} (C'_i C''_j - C'_j C''_i)^2}$$

High curvature indicates a sharp turn in semantic reasoning – a signal for uncertainty and the need for human oversight.

## Confidence from Knot Density

Confidence in a region is derived from the density of control points within a radius `r` (here `0.1` in normalised semantic space):
$$\text{confidence} = 1 - \frac{1}{1 + \frac{N_{\text{nearby}}}{5}}$$

where `N_nearby` is the number of control points within radius `r`. More nearby knots imply higher fidelity and thus higher confidence.

confidence.

## License

Proprietary – Conduit Impact Capital. For academic and research use only.

*This API reference is a living document. For the latest version, please consult the internal GeoJS repository.*